

2 - Introduction to NumPy

December 12, 2025

1 2 - Introduction to NumPy

1.1 Textbook Part II. Chapters 4 and 5.

NumPy (**N**umerical **P**ython) provides an efficient interface to store and operate on dense data buffers through **arrays**.

In some ways, NumPy arrays work like Python's built-in **lists**. However, NumPy arrays provide much more efficient operations, especially for large volumes of data.

NumPy arrays form the core of the entire data science and machine learning ecosystem.

To install NumPy using **pip**:

```
[ ]: %pip install numpy
```

To start using NumPy, we will use the **import** directive, and we will assign to NumPy the alias **np** to make it easier to refer to it in the code.

```
[ ]: import numpy as np
```

Now we can use NumPy using the **np** alias. The following line displays NumPy's built-in documentation using the **?** operator.

```
[47]: np?
```

```
Type:          module
String form: <module 'numpy' from '/Users/mgouveag/shared/courses/CSC
8000/Lecture Notebooks/Lectures/.venv/lib/python3.13/site-
packages/numpy/__init__.py'>
File:          ~/shared/courses/CSC 8000/Lecture
Notebooks/Lectures/.venv/lib/python3.13/site-packages/numpy/__init__.py
Docstring:
NumPy
=====
```

Provides

1. An array object of arbitrary homogeneous items
2. Fast mathematical operations over arrays
3. Linear Algebra, Fourier Transforms, Random Number Generation

How to use the documentation

Documentation is available in two forms: docstrings provided with the code, and a loose standing reference guide, available from the NumPy homepage <<https://numpy.org>>`\_`.

We recommend exploring the docstrings using IPython <<https://ipython.org>>`\_`, an advanced Python shell with TAB-completion and introspection capabilities. See below for further instructions.

The docstring examples assume that `numpy` has been imported as ``np``::

```
>>> import numpy as np
```

Code snippets are indicated by three greater-than signs::

```
>>> x = 42
>>> x = x + 1
```

Use the built-in ``help`` function to view a function's docstring::

```
>>> help(np.sort)
... # doctest: +SKIP
```

For some objects, ``np.info(obj)`` may provide additional help. This is particularly true if you see the line "Help on ufunc object:" at the top of the help() page. Ufuncs are implemented in C, not Python, for speed. The native Python help() does not know how to view their help, but our np.info() function does.

Available subpackages

lib
 Basic functions used by several sub-packages.
random
 Core Random Tools
linalg
 Core Linear Algebra Tools
fft
 Core FFT routines
polynomial
 Polynomial tools
testing
 NumPy testing tools
distutils
 Enhancements to distutils with support for
 Fortran compilers support and more (for Python <= 3.11)

Utilities

```
-----  
test  
    Run numpy unittests  
show_config  
    Show numpy build configuration  
__version__  
    NumPy version string
```

Viewing documentation using IPython

Start IPython and import `numpy` usually under the alias `np`: `import numpy as np`. Then, directly past or use the `%cpaste` magic to paste examples into the shell. To see which functions are available in `numpy`, type `np.<TAB>` (where `<TAB>` refers to the TAB key), or use `np.*cos*<ENTER>` (where `<ENTER>` refers to the ENTER key) to narrow down the list. To view the docstring for a function, use `np.cos?<ENTER>` (to view the docstring) and `np.cos??<ENTER>` (to view the source code).

Copies vs. in-place operation

Most of the functions in `numpy` return a copy of the array argument (e.g., `np.sort`). In-place versions of these functions are often available as array methods, i.e. `x = np.array([1,2,3]); x.sort()`. Exceptions to this rule are documented.

1.2 More on Python data types

Python is implemented in C. This means every Python object is a C structure, containing not only the object's value but other information as well.

An integer in Python, such as `x=1000` stores not only the integer value, but also a compound C structure.

This means that an integer in Python has a memory overhead not present in lower-level languages such as C.

A similar reasoning can be applied to Python's `list`. The standard `list` in Python is dynamic-typed, meaning it is capable of storing objects of different types (integers, strings, other lists, etc.). This also incurs a memory overhead that is not necessary when all objects are of the same type.

1.2.1 Fixed-type arrays in Python

Python offers a built-in `array` module (since Python 3.3) that can be used to create dense arrays of a single type:

```
[ ]: import array

L = list(range(10)) # Create a list of values
A = array.array('i', L) # Create an array of integers ('i') using the elements
    ↪ in L
print(A)
```

1.2.2 Fixed-type arrays using NumPy

While Python's `array` provides an efficient interface to handle vector data, NumPy's `ndarray` object provides much more functionality to them, such as array arithmetic, filtering, broadcasting, and more. For that reason, we will focus on `ndarray` instead of the built-in `array` objects.

1.2.3 Creating arrays from Python lists

```
[48]: import numpy as np

np.array([1, 4, 2, 5, 3])
```

```
[48]: array([1, 4, 2, 5, 3])
```

The above code infers that the data type to be used for the array is **integer**.

We can enforce a particular data type using the `dtype` parameter:

```
[ ]: np.array([1, 4, 2, 5, 3], dtype=np.float32)
```

This creates an array of 32-bit precision floating-point values.

Nested lists are converted into 2D NumPy arrays:

```
[49]: np.array([[1,2,3], [4,5,6], [7,8,9]])
```

```
[49]: array([[1, 2, 3],
           [4, 5, 6],
           [7, 8, 9]])
```

1.2.4 Creating arrays from scratch

NumPy has a number of routines to help creating arrays from scratch. Here are a few examples:

```
[50]: ## Create an array of 10 integer elements filled with 0s

np.zeros(10, dtype=int)
```

```
[50]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
[52]: ## Create a 3x5 array filled with 1s

np.ones((3, 5), dtype=float)
```

```
[52]: array([[1., 1., 1., 1., 1.],
           [1., 1., 1., 1., 1.],
           [1., 1., 1., 1., 1.]])
```

```
[53]: ## Create a 3x5 array filled with the value 3.14

np.full((3, 5), 3.14)
```

```
[53]: array([[3.14, 3.14, 3.14, 3.14, 3.14],
           [3.14, 3.14, 3.14, 3.14, 3.14],
           [3.14, 3.14, 3.14, 3.14, 3.14]])
```

```
[55]: ## Create an array filled with a sequence starting a 0, ending at 20, stepping
      ↪ by 2

np.arange(0, 21, 2)
```

```
[55]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
[56]: ## Create an array of five values evenly spaced between 0 and 1

np.linspace(0, 1, 5)
```

```
[56]: array([0.   , 0.25, 0.5  , 0.75, 1.   ])
```

```
[ ]: ## Create an identity matrix of dimensions 4x4.

np.eye(4)
```

```
[ ]: ## Create an array of 20 integers initialized with garbage values (empty).

np.empty(20, dtype=int)
```

1.2.5 NumPy standard data types

Remember: NumPy arrays contain values of a *single type*. Some of the common types we will use are:

- `np.int64` - 64-bit integer
- `np.int32` - 32-bit integer
- `np.int16` - 16-bit integer
- `np.float64` - 64-bit floating point
- `np.float32` - 32-bit float
- `np.float16` - 16-bit float

1.3 The basics of NumPy arrays

Next, we will cover some of the basic array manipulation concepts:

- Array attributes

- Indexing arrays
- Array slicing
- Array reshaping
- Array joining and splitting

1.3.1 Array attributes

To illustrate array attributes, we will define 3 randomly generated arrays using NumPy's random utility:

```
[57]: import numpy as np

rng = np.random.default_rng(seed=1701) # define a random number generator

x1 = rng.integers(10, size=6) # integers up to 10
x2 = rng.integers(10, size=(3,4)) # 2d array
x3 = rng.integers(10, size=(3, 4, 5)) # 3d array
```

Each of the above arrays has attributes including `ndim`, `shape`, `size` and `dtype`. Let's take a look at them:

```
[58]: print(f"x3 ndim: {x3.ndim}")
print(f"x3 shape: {x3.shape}")
print(f"x3 size: {x3.size}")
print(f"dtype: {x3.dtype}")
```

```
x3 ndim: 3
x3 shape: (3, 4, 5)
x3 size: 60
dtype: int64
```

Quiz - What will be the output of each of the following?

1. `x1.ndim`
2. `x2.size`
3. `x1.shape`

1.3.2 Array Indexing

Array indexing works in a very similar fashion to Python lists. In a one-dimensional array, index `i` corresponds to the `i`-th element of the array, counting from zero.

```
[ ]: print(x1[0])
print(x1[4])
```

Use negative indices to index from the end of the array:

```
[ ]: print(x1[-1])
print(x1[-2])
```

In a **multidimensional array**, items can be accessed using a comma-separated (row, column) tuple:

```
[59]: print(x2[0, 0]) # Get the element in the first row and first column
      print(x2[2, 0]) # Get the element n row 2, first column
      print(x2[2, -1])
```

```
3
0
9
```

Modifying: values can be changed using the index notation:

```
[60]: x2[0, 0] = 12
      print(x2)
```

```
[[12  1  3  7]
 [ 4  0  2  3]
 [ 0  0  6  9]]
```

1.3.3 Array slicing

Slicing of NumPy arrays are similar to the slicing of Python lists. The basic syntax is `x[start:end:step]`.

Slicing one-dimensional arrays

```
[61]: print(x1)
```

```
[9 4 0 3 8 6]
```

```
[ ]: print(x1[:3]) # Get the first three elements
```

```
[ ]: print(x1[1:4]) # Get element indices 1 through 3
```

```
[ ]: print(x1[::2]) # Get every second element
```

When `step` is *negative*, the slicing moves backward. We can use this to easily reverse an array:

```
[ ]: print(x1)
      print(x1[::-1]) # All elements, reversed
```

Slicing multidimensional arrays To slice multidimensional arrays, we use slices separated by commas:

```
[62]: print(x2)
```

```
[[12  1  3  7]
 [ 4  0  2  3]
 [ 0  0  6  9]]
```

```
[63]: print(x2[:2, :3]) # First two rows and first three columns
```

```
[[12  1  3]
 [ 4  0  2]]
```

```
[64]: print(x2[:3, ::2]) # first three rows, every second column
```

```
[[12  3]
 [ 4  2]
 [ 0  6]]
```

```
[65]: print(x2[::-1, ::-1]) # All rows and columns, reversed
```

```
[[ 9  6  0  0]
 [ 3  2  0  4]
 [ 7  3  1 12]]
```

We can use slicing combined with indexing to access a single row or column of a multidimensional array:

```
[66]: print(x2[:, 0]) # Get the first column of x2
```

```
[12  4  0]
```

```
[68]: print(x2[2, :]) # Get the third row of x2
print(x2[2])
```

```
[0 0 6 9]
[0 0 6 9]
```

Slices are no-copy views NumPy array slices are *views* of the original array, rather than *copies* of data. This means any changes to the slices **also affect the original array**. Consider the following:

```
[69]: print(x2)
```

```
[[12  1  3  7]
 [ 4  0  2  3]
 [ 0  0  6  9]]
```

```
[70]: x2_sub = x2[:2, :2]
print(x2_sub)
```

```
[[12  1]
 [ 4  0]]
```

Let us modify `x2_sub` and see what happens to `x2`:

```
[71]: x2_sub[0, 0] = 99
print(x2)
```

```
[[99  1  3  7]
 [ 4  0  2  3]
 [ 0  0  6  9]]
```

Creating copies of arrays Using views can be an efficient way of accessing sub arrays. However, you might want to create a real copy of the array after slicing. To do that, you can use the `copy` method:

```
[72]: x2_sub_copy = x2[:2, :2].copy()
      print(x2_sub_copy)
```

```
[[99  1]
 [ 4  0]]
```

Changes made to `x2_sub_copy` will **not be reflected upon `x2`**.

1.3.4 Reshaping arrays

Array reshaping can be used to change the shape of your array, such as turning a 1D array into a 2D one. This is done using the `reshape` method. For example, assume you want to put the numbers 1 through 9 in a 3 x 3 grid:

```
[73]: grid = np.arange(1, 10).reshape(3, 3)
      print(grid)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Note that the initial array size **must match the size of the reshaped array**.

1.3.5 Array concatenation and splitting

NumPy provides tools to combine arrays into one, or split arrays into many.

Array concatenation Joining two arrays in NumPy can be accomplished in many ways, using: `np.concatenate`, `np.vstack`, `np.hstack`.

`np.concatenate` takes a tuple or a list of arrays and produces a single array that concatenates them:

```
[74]: x = np.array([1,2,3])
      y = np.array([3,2,1])

      print(np.concatenate([x, y])) # concatenate arrays x and y
```

```
[1 2 3 3 2 1]
```

You can concatenate multiple arrays at once:

```
[75]: z = np.array([99, 99, 99])
      print(np.concatenate([x, y, z]))
```

```
[ 1  2  3  3  2  1 99 99 99]
```

For working with arrays of mixed dimensions, you can use `np.vstack` to stack arrays vertically, or `np.hstack` to stack arrays horizontally:

```
[ ]: grid = np.array([[1,2,3],
                      [4,5,6]]) # create a 2D array
```

```
[ ]: # Stack grid and array x vertically

print(np.vstack([x, grid]))
```

```
[ ]: # Stack grid and y horizontally

y = np.array([[99], [99]])
print(np.hstack((grid, y)))
```

Array splitting To split an array, you can use function `np.split(a, indices)`. Where `a` is an input array, and `indices` is a list of indices where to split. For example:

```
[ ]: x = [1, 2, 3, 99, 99, 3, 2, 1]
x1, x2, x3 = np.split(x, [3, 5])
print(x1, x2, x3)
```

The above example splits array `x` at indices 3 and 5, produces **three** sub arrays as output.

Similarly, you can split arrays horizontally or vertically with `np.hsplit` and `np.vsplit`, respectively:

```
[ ]: grid = np.arange(16).reshape((4,4))

upper, lower = np.vsplit(grid, [2])

print(upper)
print(lower)
```

```
[ ]: left, right = np.hsplit(grid, [2])
print(left)
print(right)
```

1.4 Testing the performance of NumPy

Let us compare the running time of two programs that create very long lists/arrays. The first uses a Python list, the second uses NumPy's `arange`.

```
[76]: %%timeit
l = []
for i in range(1000000):
    l.append(i)
```

19.3 ms ± 71.8 s per loop (mean ± std. dev. of 7 runs, 10 loops each)

```
[77]: %%timeit
x = np.arange(1000000)
```

480 s $\pm$ 4.42 s per loop (mean $\pm$ std. dev. of 7 runs, 1,000 loops each)